

SynAgent: Round-Trip Retrosynthesis Validation

Design Document & Implementation Proposal

Aaditya Sanil · Head-Gordon Group, LBNL · June 2026

1. Problem Statement

SynLlama produces retrosynthetic pathways of the form **product** → **reactants**. A core question in the SynAgent pipeline is: are these retrosynthetic steps chemically self-consistent? Naively reversing the tree (reactants → product) to obtain a synthesis pathway is insufficient because some reactions are **not microscopically reversible** — a valid retrosynthetic disconnection does not guarantee the forward reaction proceeds under accessible conditions.

The existing *validation.py* agent uses RDKit reaction template execution and SMILES validity checks, but does not assess whether SynLlama's own internal representation of the chemistry is self-consistent across stochastic samples.

2. Proposed Solution: Round-Trip Consistency Check

The key insight is that SynLlama is both the **generator** and the **validator**. If a retrosynthetic step is chemically meaningful, feeding the reactants back into SynLlama should recover the original product with high probability across many stochastic samples.

2.1 Core Algorithm

Step	Action	Cost
1	SynLlama generates: product → [reactant_1, reactant_2]	1x LLM call
2	Feed reactant_1.reactant_2 back into SynLlama retro × N	N x LLM calls
3	Count how many of N runs recover original product	—
4	confidence = hit_count / N	—
5	If confidence < threshold → flag for agent validation	1x agent call (only if flagged)

2.2 Why N=50-100 samples?

SynLlama uses temperature-based sampling, so each call produces a different top-k set of retrosynthetic suggestions. Running N=50–100 calls gives a **distribution** over possible retrosynthetic paths from the reactants. A high hit rate (e.g. 70/100) indicates the step is a strong, unambiguous reaction. A low hit rate (e.g. 15/100) indicates chemical ambiguity — the reactants could plausibly lead to many different products, making the step unreliable.

Crucially, SynLlama inference is **very cheap compared to MCP/agent calls**. 100 LLM calls at ~\$0.001 each = \$0.10 per pathway step. One agent tool call with ASKCOS or external validation can cost orders of

magnitude more in latency and compute. This makes round-trip validation an ideal **pre-filter**.

3. Pipeline Architecture

Stage	Component	Output
1. Generation	SmileyLlama → SynLlama	Retro pathway (product → reactants)
2. RDKit check	rdkit_validate_step()	Binary pass/fail on template execution
3. Round-trip	RoundTripValidator (N samples)	confidence score per step [0.0–1.0]
4. Filter	confidence >= threshold (0.5)	Pass → synthesis pathway; Fail → flagged
5. Agent review	SynAgent (expensive)	Only runs on flagged steps

4. SynLlama CSV Format (synllama-raw-output.csv)

Each row contains a target SMILES, sampling parameters, and a JSON response blob with reactions and building blocks. Example row (truncated):

```
smiles,sampling_params,response
Cc1csc(NC(=O)... )n1, medium, {
  "reactions": [{
    "reaction_number": 1,
    "reaction_template": "<rxn>[Cl,OH...][C...]=O.[N...]>>[N]-[C]=[O]</rxn>",
    "reactants": ["Cc1csc(N)n1", "O=C(O)c1cccnc1N1CCOCC1"],
    "product": "Cc1csc(NC(=O)c2cccnc2N2CCOCC2)n1"
  }],
  "building_blocks": ["<bb>Cc1csc(N)n1</bb>", "<bb>O=C(O)...</bb>"]
}
```

The parser strips <rxn> and <bb> tags, canonicalizes SMILES via RDKit, and constructs ReactionStep objects for each reaction entry.

5. Pydantic Model Design

```
class ReactionStep(BaseModel):
    step_index: int
    product: str
    reactants: list[str]
    reaction_template: str | None = None
    sampling_param: str | None = None
    # Populated after validation:
    hit_count: int = 0
    n_samples: int = 0
    confidence: float = 0.0 # hit_count / n_samples
    flagged: bool = False # True if confidence < threshold
    rdkit_valid: bool = False # RDKit forward execution check
```

```

class ValidationResult(BaseModel):
    molecule_smiles: str
    steps: list[ReactionStep]
    overall_confidence: float = 0.0
    passed: bool = False
    threshold: float = 0.5

class RoundTripValidator(BaseModel):
    n_samples: int = 50
    threshold: float = 0.5

    async def validate_step(step, synllama_fn) -> ReactionStep: ...
    async def validate_pathway(steps, synllama_fn) -> ValidationResult: ...
    def summary(result) -> dict: ...

```

6. SynAgent Integration

The round-trip validator integrates into *validation.py* as a new **@agent.tool_plain** alongside the existing *is_valid_smiles* and *is_valid_reaction* tools. The agent decides when to call it based on the system prompt — typically after initial SMILES validation passes.

```

@agent.tool_plain
async def roundtrip_validate_step(
    product: str,
    reactants: list[str],
    reaction_template: str,
    n_samples: int = 50,
) -> dict:
    """
    Stochastic round-trip validation: feeds reactants back into SynLlama
    N times and checks what fraction recover the original product.
    Returns confidence score and flagged status.
    High confidence (>0.5) = step is self-consistent.
    Low confidence (<0.5) = escalate to full agent validation.
    """
    step = ReactionStep(step_index=1, product=product,
                        reactants=reactants, reaction_template=reaction_template)
    validator = RoundTripValidator(n_samples=n_samples, threshold=0.5)
    validated = await validator.validate_step(step, synllama_fn=YOUR_SYNLlama_FN)
    return {
        "confidence": validated.confidence,
        "rdkit_valid": validated.rdkit_valid,
        "flagged": validated.flagged,
        "hits": f"{validated.hit_count}/{validated.n_samples}",
    }

```

7. Cost vs. Accuracy Tradeoff

N samples	Est. cost/step	Confidence resolution	Recommended for
10	~\$0.01	±31% (high variance)	Quick smoke test
50	~\$0.05	±14%	Standard pipeline run
100	~\$0.10	±10%	PI presentation / paper
1 agent call	~\$0.50–2.00	Single deterministic result	Flagged steps only

Recommendation: use N=50 for standard CACHE Challenge pipeline runs. Use N=100 when generating results for the paper or PI review. Reserve agent calls for steps with confidence below 0.5.

8. Relationship to ASKCOS Forward Validation

Round-trip SynLlama validation is a **self-consistency check** — it tests whether SynLlama's own model agrees with itself across samples. This is distinct from ASKCOS forward prediction, which is an **independent external oracle** trained on USPTO/Reaxys reaction data.

Method	What it checks	Speed	Independence
RDKit template exec	Template applies to reactants	Fast (ms)	High — rule-based
Round-trip SynLlama (N=50)	SynLlama self-consistency	Medium (N x LLM)	Low — same model
ASKCOS forward predictor	External ML oracle	Slow (HTTP API)	High — different model/data
Full SynAgent	Agentic reasoning + tools	Slowest	High — multi-step reasoning

Ideal validation cascade: RDKit (free) → Round-trip SynLlama (cheap) → ASKCOS forward (moderate) → SynAgent (expensive, only for hard cases).

9. Next Steps

1. Wire real SynLlama inference function into `_mock_synllama` placeholder in `roundtrip_validation.py`
2. Run on `synllama-raw-output.csv` and `synllama-raw-valid.csv` to get baseline confidence distributions
3. Compare confidence scores between valid vs. failed molecules — expect valid CSV to have higher mean confidence
4. Tune threshold based on empirical distribution (start at 0.5, adjust based on precision/recall)
5. Add `roundtrip_validate_step` tool to `validation.py` agent and update system prompt
6. Consider adding ASKCOS forward API call as third stage for flagged steps if budget allows

References

- Tu et al. (2025). ASKCOS: an open source software suite for synthesis planning. arXiv:2501.01835

- Genheden et al. (2020). AiZynthFinder: a fast, robust and flexible open-source software for retrosynthetic planning. J. Cheminform.
- Cavanagh et al. SynLlama (internal). Head-Gordon Group, LBNL.
- Coley et al. (2019). RDChiral: An RDKit Wrapper for Handling Stereochemistry in Retrosynthetic Template Extraction. J. Chem. Inf. Model.